

# src/basis\_functions.cpp — Function-by-Function Explanation

A-Phi Solver project — code reference

September 2026

## Contents

|                                                                                 |   |
|---------------------------------------------------------------------------------|---|
| Overview                                                                        | 1 |
| invert4x4                                                                       | 1 |
| compute_tet_geometry                                                            | 2 |
| evaluate_barycentric                                                            | 2 |
| whitney_edge_value                                                              | 3 |
| whitney_edge_curl                                                               | 3 |
| p2_nodal_value                                                                  | 3 |
| p2_nodal_gradient                                                               | 4 |
| Why the two element families don't fully match (documented, accepted trade-off) | 4 |

## Overview

This file implements the element-level basis functions locked in by docs/FORMULATION.md Sec. 5.1: the classic **first-order Whitney/Nédélec tetrahedral edge element** for the vector potential **A** (6 degrees of freedom per tet, one per edge), and the **second-order (P2, 10-node) Lagrange nodal element** for the scalar potential **Phi**. Both basis families are built on top of a shared piece of per-tet geometry: the tetrahedron's four barycentric coordinates and their (constant) gradients, computed once per tet by `compute_tet_geometry`.

There are seven functions in the file: one internal helper (`invert4x4`, in an anonymous namespace, not visible outside this file) and six functions declared in `include/aphi_solver/basis_functions.hpp` and used throughout the rest of the codebase. They are covered below in the order they appear in the file, which is also a natural dependency order — each later function builds on the ones before it.

All seven functions are exercised by `tests/test_basis_functions.cpp` (350/350 checks passing as of this writing), which is the practical way to confirm this explanation matches the code's actual behavior.

---

## invert4x4

```
std::array<std::array<double, 4>, 4> invert4x4(std::array<std::array<double, 4>, 4> m)
```

A general-purpose 4x4 matrix inverse, computed by Gauss-Jordan elimination with partial pivoting. It is a private implementation detail (declared in an unnamed namespace, so it has no external linkage) used only by `compute_tet_geometry` below.

**How it works:** it starts with the identity matrix `inv` and applies the same row operations to both `m` and `inv` simultaneously. For each column, it picks the row with the largest-magnitude entry in that column as the pivot (partial pivoting — this keeps the elimination numerically stable even when the input tetrahedron’s vertex coordinates aren’t a convenient size), swaps it into place, normalizes that row so the pivot becomes 1, and then subtracts multiples of it from every other row so the rest of that column becomes 0. After all four columns are processed this way, `m` has become the identity and `inv` has become `m`’s inverse.

**Why it throws:** if the largest available pivot in a column is smaller than  $1e-14$ , the matrix is treated as singular and the function throws `std::runtime_error`. For the specific matrix this is used on (built from a tet’s 4 vertex coordinates — see `compute_tet_geometry`), a singular matrix means the 4 vertices are coplanar, i.e. the tetrahedron has zero volume and is degenerate. This is a mesh-quality problem the caller should know about immediately rather than silently getting garbage geometry.

---

## compute\_tet\_geometry

`TetGeometry compute_tet_geometry(const Mesh& mesh, int t)`

Computes the affine (linear) formula for each of a tetrahedron’s four barycentric coordinate functions `L0..L3`, and packages them into a `TetGeometry` struct (`a[4]`, `grad_L[4]`, `volume`). Every other function in this file takes a `TetGeometry` as an input rather than recomputing this — it only needs to be done once per tet, since `grad_L` is constant over the whole element (the barycentric coordinates are linear functions of position).

**The math:** each barycentric coordinate has the form  $L_i(x,y,z) = a[i] + \text{grad\_L}[i] \cdot (x,y,z)$ , and by definition  $L_i$  must equal 1 at the tet’s own vertex `i` and 0 at the tet’s other three vertices. Writing a 4x4 matrix `M` whose row `j` is `[1, x_j, y_j, z_j]` (the homogeneous coordinates of vertex `j`), this Kronecker-delta condition becomes the matrix equation  $M \cdot v_i = e_i$ , where  $v_i = [a_i, \text{grad\_L}_i]$  is what we want to solve for and  $e_i$  is the `i`-th standard basis vector. Since this holds for all four `i` at once, the four unknown vectors `v_0..v_3` are exactly the four columns of  $M^{-1}$ . The function builds `M` from the tet’s vertex positions (via `mesh.tets[t]` and `mesh.nodes`), inverts it with `invert4x4`, and reads `a[i]` and `grad_L[i]` off column `i` of the result.

This approach is deliberately generic: it makes no assumption about vertex ordering or orientation, unlike shortcuts that only work for a specific reference tetrahedron. It works identically for any non-degenerate tet.

**The volume field:** `TetGeometry::volume` is set to the absolute value of `mesh.signed_tet_volume(t)` — the unsigned tet volume, reserved for quadrature weight scaling once numerical integration is needed (Phase 04), not used by any of the basis-evaluation functions below.

---

## evaluate\_barycentric

`std::array<double, 4> evaluate_barycentric(const TetGeometry& g, const Vec3& p)`

Evaluates all four barycentric coordinates at an arbitrary Cartesian point `p`, using the affine coefficients `g.a` and `g.grad_L` that `compute_tet_geometry` already computed for this tet. This is a direct plug-in of `p` into  $L_i(p) = a[i] + \text{grad\_L}[i] \cdot p$  for each of the four coordinates — no matrix work needed at this point, since that was already done once per tet.

Every basis-function evaluator below (`whitney_edge_value`, `p2_nodal_value`, `p2_nodal_gradient`) takes an `L` array as input rather than a raw point `p`, so this function is the standard way callers convert a physical evaluation point into the barycentric coordinates those functions expect — for instance, at a quadrature point, or (as in the test suite) at a specific mesh vertex or edge midpoint to check a basis function’s value there.

---

## whitney\_edge\_value

Vec3 whitney\_edge\_value(const TetGeometry& g, int local\_edge, const std::array<double, 4>& L)

Evaluates the first-order Whitney/Nédélec edge basis function for **A** at a point whose barycentric coordinates are **L**. local\_edge (0..5) selects which of the tet's six edges this basis function belongs to, via kTetLocalEdgeVerts[local\_edge], which gives the pair of local vertex indices (vi, vj) at that edge's two ends.

**The formula**, implemented directly as written:

$$N_{ij}(p) = L_i(p) * \text{grad}(L_j) - L_j(p) * \text{grad}(L_i)$$

This is the classical Whitney 1-form for edge (i, j), sourced in this project from Bossavit (1998) Ch. 5 and cross-checked against the user's own prior implementation (BasisFunction.docx / EdModel.cpp). Physically, one basis function like this is associated with each of the tet's six edges, and the DOF it represents is the tangential line integral of **A** along that edge ( $\int_{\text{edge}} \mathbf{A} \cdot d\mathbf{l}$ ) — this is what makes it a first-order (“lowest order”) edge element: 6 DOF per tet, matching the 6 edges, rather than a node-based count.

tests/test\_basis\_functions.cpp verifies this formula's defining property directly: integrating  $N_j$ 's tangential component along edge m of the same tet gives exactly 1 when  $m == j$  and exactly 0 for every other edge — the Kronecker-delta/dual-basis property that edge elements are built to satisfy.

---

## whitney\_edge\_curl

Vec3 whitney\_edge\_curl(const TetGeometry& g, int local\_edge)

Returns the curl of the same edge basis function  $N_{ij}$ , which for a first-order Whitney element is **constant** over the whole tetrahedron — this function takes no evaluation point p because none is needed.

**The formula:**

$$\text{curl}(N_{ij}) = 2 * \text{grad}(L_i) \times \text{grad}(L_j)$$

This is a direct consequence of differentiating the whitney\_edge\_value formula: since  $\text{grad}(L_i)$  and  $\text{grad}(L_j)$  are themselves constant over the tet, and the curl of  $L_i * \text{grad}(L_j)$  works out (via a standard vector identity) to  $\text{grad}(L_i) \times \text{grad}(L_j)$ , the two terms of  $N_{ij}$  end up adding rather than cancelling, giving the factor of 2 above.

This constant curl is the reason first-order Whitney elements are so convenient for magnetostatic/eddy-current problems:  $\mathbf{B} = \text{curl}(\mathbf{A})$  is piecewise constant per tet, with no quadrature needed to evaluate it inside an element. tests/test\_basis\_functions.cpp cross-checks this closed form against a finite-difference numerical curl of the actual whitney\_edge\_value field, as an independent confirmation the formula was transcribed correctly.

---

## p2\_nodal\_value

double p2\_nodal\_value(int local\_node, const std::array<double, 4>& L)

Evaluates the second-order (P2) Lagrange nodal basis function for **Phi** at a point with barycentric coordinates **L**. A tet's P2 element has 10 nodes: local\_node 0..3 are the tet's own 4 vertices, and 4..9 are the 6 edge midpoints (in the same order as kTetLocalEdgeVerts, so local\_node - 4 is directly the local edge index whose midpoint that node sits on).

**Two cases, matching the standard P2 tetrahedron construction** (verified in this project against Jin (2014), Ch. 5, Eq. 5.52 / Fig. 5.3):

- **Vertex node** (`local_node < 4`):  $N_i = (2 \cdot L_i - 1) \cdot L_i$ . This is 1 at vertex  $i$  itself and 0 at every other vertex and every edge midpoint.
- **Edge-midpoint node** (`local_node >= 4`): reading off the edge's two endpoint indices ( $v_0, v_1$ ) from `kTet - LocalEdgeVerts[local_node - 4]`, the function is  $N = 4 \cdot L_{v_0} \cdot L_{v_1}$ . This is 1 at that edge's own midpoint and 0 everywhere else among the 10 nodes.

`tests/test_basis_functions.cpp` checks exactly this Kronecker-delta property at all 10 nodes, and separately checks that the 10 values always sum to 1 at an arbitrary point (partition of unity) — both standard correctness properties of a nodal finite-element basis.

---

## p2\_nodal\_gradient

`Vec3 p2_nodal_gradient(const TetGeometry& g, int local_node, const std::array<double, 4>& L)`

Returns the gradient of the same P2 nodal basis function, at the point whose barycentric coordinates are  $L$ . Unlike `whitney_edge_curl` above, this is **not** constant over the tet — the P2 basis functions are quadratic in position, so their gradients are linear (they genuinely depend on  $L$ , hence on the evaluation point).

**Vertex node** (`local_node < 4`): since  $N_i = 2 \cdot L_i^2 - L_i$ , the one-variable chain rule gives  $dN_i/dL_i = 4 \cdot L_i - 1$ , and because  $L_i$  itself is an affine function of position with constant gradient  $\text{grad}(L_i)$ , the spatial gradient is  $\text{grad}(N_i) = (4 \cdot L_i - 1) \cdot \text{grad}(L_i)$ .

**Edge-midpoint node** (`local_node >= 4`): since  $N = 4 \cdot L_{v_0} \cdot L_{v_1}$  is a product of two coordinates, the product rule gives  $\text{grad}(N) = 4 \cdot (L_{v_1} \cdot \text{grad}(L_{v_0}) + L_{v_0} \cdot \text{grad}(L_{v_1}))$ .

`tests/test_basis_functions.cpp` cross-checks both formulas against a finite-difference numerical gradient of `p2_nodal_value` at a sample point, for all 10 nodes.

---

## Why the two element families don't fully match (documented, accepted trade-off)

Worth noting for anyone reading this file alongside `docs/FORMULATION.md` Sec. 5.1: because **A** stays first-order while **Phi** is second-order, `grad(p2_nodal_value(...))` is **not** exactly representable as a linear combination of `whitney_edge_value(...)` terms in general — a from-scratch proof of this (comparing the symmetric Jacobian of a P2 gradient's linear part against the always-skew-symmetric Jacobian of a first-order Nédélec function) is recorded in `docs/FORMULATION.md`. This caps the accuracy of the **A-Phi** coupling term at first order. It was a deliberate, informed decision, not an oversight in this file's implementation, and it does not affect tree-cotree gauging (Phase 03), which operates purely on **A**'s own edge space.